SERVICIO NACIONAL DE APRENDIZAJE - SENA
Tecnología en Análisis y Desarrollo de Software (ADSO) – Ficha 3186698

Informe técnico de plan de trabajo para construcción de software

Herramientas y tecnologías de versionamiento

Proyecto: sistema de predicción y análisis estadístico de fútbol

Evidencia GA7-220501096-AA1-EV01

Aprendiz: Juan Sebastián Garibello Vizcaíno

2026

Introducción

Durante la construcción de un software, el código, las bases de datos, las interfaces y la documentación cambian de forma constante. Sin un control ordenado de esos cambios es fácil perder trabajo, sobrescribir el aporte de otra persona o no poder regresar a una versión que funcionaba. El versionamiento resuelve este problema: registra cada modificación, permite trabajar en paralelo y facilita recuperar cualquier estado anterior del proyecto.

Este informe técnico presenta el plan de trabajo para la construcción del sistema de predicción y análisis estadístico de fútbol, y sustenta la selección de las herramientas y tecnologías de versionamiento que se usarán: el sistema de control de versiones, la plataforma de alojamiento del repositorio, las herramientas de apoyo y la estrategia de ramas y de mensajes de cambio.

Objetivo

Seleccionar y justificar las herramientas y tecnologías de versionamiento para el desarrollo del software del proyecto formativo, y definir el plan de trabajo con el que se aplicarán durante su construcción.

- Describir las características del software y los elementos que deben versionarse.
- Comparar las alternativas de sistemas de control de versiones y de plataformas de alojamiento.
- Definir la estrategia de ramas, la convención de mensajes de commit y la estructura del repositorio.
- Establecer un plan de trabajo por fases indicando cómo se usa el versionamiento en cada una.

1. Descripción del software a desarrollar

El proyecto es un sistema web de predicción y análisis estadístico de partidos de fútbol internacional (por ejemplo, Champions League y Premier League). Su propósito es académico y analítico: no gestiona dinero ni apuestas. El sistema estima probabilidades de victoria, empate y derrota, y los goles esperados de cada equipo, usando un modelo de regresión de Poisson.

Componente	Tecnología prevista
Base de datos	Base de datos relacional (MySQL) con tablas de ligas, equipos, partidos, estadísticas, modelos y predicciones.
Back-end y modelo estadístico	Python con FastAPI o Django; pandas y scikit-learn para el análisis y el modelo.
Front-end	HTML, CSS y JavaScript (interfaz con menú lateral, listados, filtros y simulador).
Entorno de desarrollo	Visual Studio Code.

Componente	Tecnología prevista
Documentación	Documentos Word/PDF, diagramas y scripts SQL.

2. Elementos que se deben versionar

Se versiona todo lo que se puede reconstruir a partir de texto y que es necesario para que otra persona ejecute el proyecto:

- Código fuente del back-end (Python), del modelo estadístico y del front-end (HTML, CSS y JS).
- Scripts de la base de datos (creación de tablas, restricciones y datos de prueba).
- Archivos de configuración de ejemplo (por ejemplo .env.example) y la lista de dependencias (requirements.txt).
- Documentación técnica y diagramas del proyecto.

No se versionan: contraseñas y llaves de acceso (archivo .env), entornos virtuales, archivos temporales o de caché, y datos masivos descargables. Estos elementos se excluyen mediante el archivo .gitignore.

3. Selección de herramientas de versionamiento

3.1 Sistema de control de versiones

Se compararon los tres sistemas de control de versiones más conocidos:

Criterio	Git	Subversion (SVN)	Mercurial
Modelo	Distribuido: cada copia tiene todo el historial	Centralizado: el historial vive en el servidor	Distribuido
Trabajo sin conexión	Sí: se pueden hacer commits locales	Limitado	Sí
Ramas	Muy ligeras y rápidas de crear y fusionar	Más pesadas y lentas	Ligeras
Ecosistema	El más usado; integrado en editores y plataformas	Uso decreciente	Menor comunidad
Curva de aprendizaje	Media; abundante documentación	Baja	Baja a media

Decisión: se selecciona Git. Es el estándar de la industria, permite trabajar sin conexión, tiene un manejo de ramas eficiente y es compatible con todas las plataformas de alojamiento y con Visual Studio Code.

3.2 Plataforma de alojamiento del repositorio

Criterio	GitHub	GitLab	Bitbucket
Pull/Merge Requests	Pull Requests con revisión de código	Merge Requests con revisión	Pull Requests
Gestión de tareas	Issues y Projects (tablero)	Issues y tableros	Integrada con Jira
Automatización (CI/CD)	GitHub Actions	GitLab CI/CD integrado	Bitbucket Pipelines
Integración con VS Code	Muy alta (extensión oficial)	Alta	Media
Comunidad y portafolio	La mayor comunidad; sirve como portafolio profesional	Grande	Menor

Decisión: se selecciona GitHub. Ofrece repositorios remotos con copia de seguridad en la nube, Pull Requests para revisar los cambios, Issues y Projects para organizar las tareas, y GitHub Actions para automatizar pruebas más adelante. Además, permite mostrar el trabajo como portafolio profesional. Los límites y condiciones de los planes gratuitos pueden variar, por lo que deben consultarse en la documentación oficial al crear el repositorio.

3.3 Herramientas de apoyo

- Git (línea de comandos / Git Bash): herramienta base para todas las operaciones.
- Control de código fuente de Visual Studio Code: permite ver diferencias, hacer commits y cambiar de rama desde el editor.
- GitHub Desktop (opcional): cliente gráfico para quien prefiera no usar la línea de comandos.
- Conventional Commits y Versionado Semántico: convenciones para nombrar los mensajes de cambio y las versiones.

3.4 Resumen de herramientas seleccionadas

Necesidad	Herramienta	Justificación
Control de versiones	Git	Distribuido, rápido y estándar de la industria
Repositorio remoto	GitHub	Pull Requests, Issues, Actions y copia en la nube
Edición y control desde el editor	Visual Studio Code	Integración nativa con Git y GitHub
Mensajes de cambio	Conventional Commits	Historial claro y ordenado
Numeración de versiones	Versionado Semántico (MAJOR.MINOR.PATCH)	Comunica el alcance de cada entrega

4. Estrategia de ramas y convenciones

Se adopta un flujo de trabajo simple, adecuado para un equipo pequeño (GitHub Flow): la rama main siempre contiene una versión estable, y cada funcionalidad se desarrolla en una rama propia que se integra a main mediante un Pull Request.

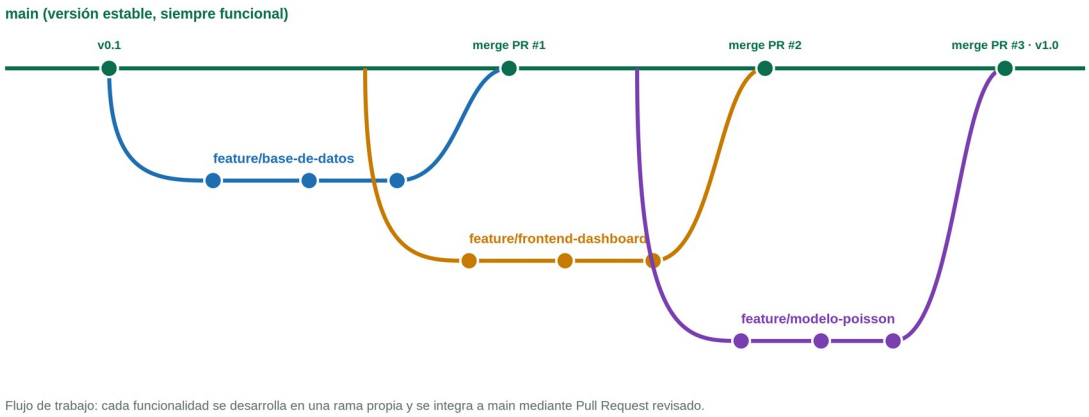


Figura 1. Flujo de ramas propuesto para el proyecto.

4.1 Nombres de ramas

- main: versión estable. No se trabaja directamente sobre ella.
- feature/nombre-corto: nueva funcionalidad (por ejemplo feature/base-de-datos).
- fix/nombre-corto: corrección de errores (por ejemplo fix/filtro-partidos).
- docs/nombre-corto: cambios de documentación.

4.2 Mensajes de commit (Conventional Commits)

Cada commit usa un tipo seguido de una descripción breve en presente:

Tipo	Uso	Ejemplo
feat	Nueva funcionalidad	feat: agregar simulador de predicción
fix	Corrección de un error	fix: corregir cálculo de probabilidad de empate
docs	Documentación	docs: actualizar diccionario de datos
style	Formato o estilos sin cambiar la lógica	style: ajustar espaciado del menú lateral
refactor	Reorganizar código sin cambiar su resultado	refactor: separar funciones del modelo Poisson
test	Pruebas	test: agregar pruebas de la función predecir

4.3 Versiones (etiquetas)

Las entregas importantes se marcan con etiquetas (tags) siguiendo el Versionado Semántico: v0.1.0 para el primer prototipo funcional, v0.2.0 al incorporar nuevas

funcionalidades, y v1.0.0 para la entrega final. Un cambio menor de corrección incrementa el tercer número (por ejemplo v1.0.1).

5. Estructura propuesta del repositorio

```
futbol-predicciones/
├── backend/          # API en Python y modelo estadístico
├── frontend/         # HTML, CSS y JS
├── database/         # scripts SQL (estructura, restricciones, datos de prueba)
├── docs/             # informes, diagramas y capturas
├── tests/            # pruebas automatizadas
├── .gitignore        # excluye .env, entornos virtuales y archivos temporales
├── .env.example      # ejemplo de configuración sin datos sensibles
├── requirements.txt  # dependencias de Python
└── README.md         # descripción y forma de ejecutar el proyecto
```

6. Flujo de trabajo con comandos básicos

Ciclo típico para desarrollar una funcionalidad:

```
git clone https://github.com/usuario/futbol-predicciones.git   # copiar el repositorio
git checkout -b feature/simulador                              # crear la rama de trabajo
git add .                                                      # preparar los cambios
git commit -m "feat: agregar simulador de predicción"          # guardar el cambio
git push -u origin feature/simulador                           # subir la rama a GitHub
# En GitHub: abrir un Pull Request, revisarlo y fusionarlo a main
git checkout main && git pull                                  # actualizar la copia local
git tag -a v0.2.0 -m "Simulador de predicción"                 # marcar la versión
git push origin v0.2.0                                         # subir la etiqueta
```

7. Plan de trabajo

El plan se organiza en fases. En cada una se indica cómo interviene el versionamiento. Las fases siguen el orden natural del proyecto y se ajustan al tiempo disponible del ciclo formativo.

N.º	Fase	Actividades principales	Uso del versionamiento
1	Análisis y planeación	Definir requisitos, alcance, herramientas y plan de trabajo	Crear el repositorio, README y tablero de tareas (Issues/Projects)
2	Diseño de la base de datos	Modelo conceptual, lógico y físico; restricciones y scripts	Rama feature/base-de-datos; commits de cada script SQL

N.º	Fase	Actividades principales	Uso del versionamiento
3	Prototipado y diseño de interfaz	Prototipo, paleta de colores, menús y usabilidad	Rama docs/prototipo con capturas y guía de estilos
4	Desarrollo del front-end	Pantallas en HTML, CSS y JS; filtros y simulador	Rama feature/frontend-dashboard; Pull Request a main
5	Desarrollo del back-end y modelo	API, conexión a la base de datos y modelo de Poisson	Rama feature/modelo-poisson; pruebas antes de fusionar
6	Pruebas e integración	Pruebas funcionales, corrección de errores e integración	Ramas fix/*; Issues para cada error encontrado
7	Documentación y entrega	Manual, informe final y versión 1.0	Etiqueta v1.0.0 y actualización del README

8. Buenas prácticas y control de riesgos

Riesgo	Medida
Pérdida de código	Repositorio remoto en GitHub y push frecuente; cada rama funciona como copia de seguridad.
Filtración de contraseñas o llaves	Uso de .env con .gitignore; solo se sube .env.example.
Conflictos al fusionar cambios	Ramas cortas, actualizar la rama con main antes del Pull Request y commits pequeños.
Código dañado en la versión estable	No se trabaja en main; toda integración pasa por Pull Request y revisión.
Historial confuso	Mensajes con Conventional Commits y etiquetas de versión.

Conclusiones

La combinación de Git como sistema de control de versiones y GitHub como plataforma de alojamiento cubre las necesidades del proyecto: permite recuperar cualquier versión, trabajar en paralelo sin perder cambios, revisar el código antes de integrarlo y conservar una copia de seguridad en la nube. El uso de Visual Studio Code facilita el trabajo diario porque integra el control de versiones en el propio editor.

La estrategia de ramas por funcionalidad, los mensajes de commit estandarizados y las etiquetas de versión dan orden al historial del proyecto. Con el plan de trabajo por fases, el versionamiento se vuelve parte de cada etapa de la construcción y no una tarea aislada, lo que reduce riesgos y mejora la calidad de la entrega final.

Referencias bibliográficas

Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book

Conventional Commits. (s.f.). Conventional Commits 1.0.0. https://www.conventionalcommits.org

GitHub. (s.f.). GitHub Docs. https://docs.github.com

Preston-Werner, T. (s.f.). Semantic Versioning 2.0.0. https://semver.org

Servicio Nacional de Aprendizaje – SENA. Componente formativo GA7-220501096, actividad AA1.